



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SEZG651/ SSZG653 Software Architectures Module 4

CS06 Documenting Software Architecture
Structures and Views



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Architectural Structures and Views

Harvinder S Jabbal
SSZG653 Software Architectures



BITS Pilani

Pilani|Dubai|Goa|Hyderabad

SEZG651/ SSZG653

Software Architectures

Module 4 -CS 06/1

August 29, 2026

SSZG653 Software Architectures



Structure & View

Structure & View



- A **view** is a representation of a coherent set of architectural elements, as written by and read by system stakeholders.
 - It consists of a representation of a set of elements and the relations among them.
- A **structure** is the set of elements itself, as they exist in software or hardware.

example



- A **module structure** is the set of the system's modules and their organization.
- A **module view** is the representation of that structure, as documented by and used by some system stakeholders.
- These terms are often used interchangeably, but we will adhere to these definitions.



Architectural structures

Module structures.



- Here the elements are modules, which are units of implementation.
- Modules represent a code-based way of considering the system.
- They are assigned areas of functional responsibility.
- There is less emphasis on how the resulting software manifests itself at runtime.
- Module structures allow us to answer questions such as
 - What is the primary functional responsibility assigned to each module?
 - What other software elements is a module allowed to use?
 - What other software does it actually use?
 - What modules are related to other modules by generalization or specialization (i.e., inheritance) relationships?

Component-and-connector structures.



- Here the elements are
 - runtime components (which are the principal units of computation) and
 - connectors (which are the communication vehicles among components).
- Component-and-connector structures help answer questions such as
 - What are the major executing components and how do they interact?
 - What are the major shared data stores?
 - Which parts of the system are replicated?
 - How does data progress through the system?
 - What parts of the system can run in parallel?
 - How can the system's structure change as it executes?

Allocation structures.



- Allocation structures show the relationship between
 - the software elements and
 - the elements in one or more external environments in which the software is created and executed.
- They answer questions such as
 - What processor does each software element execute on?
 - In what files is each element stored during development, testing, and system building?
 - What is the assignment of software elements to development teams?



architectural design

3 structures

3 broad decision types

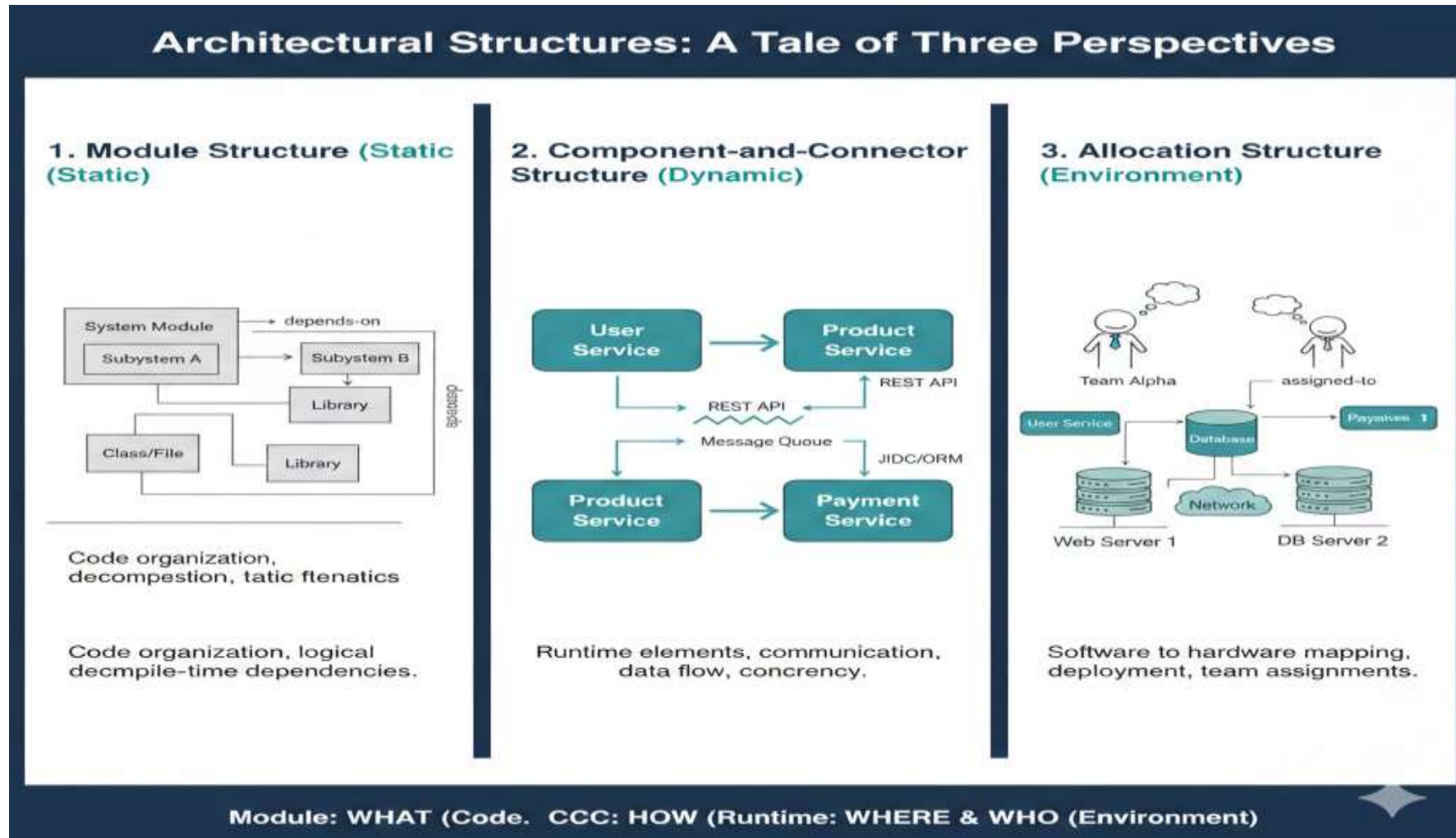


- How is the system to be structured as a set of code units (modules)?
- How is the system to be structured as a set of elements that have
 - runtime behavior (components) and
 - interactions (connectors)?
- How is the system to relate to nonsoftware structures in its environment
 - (i.e., CPUs, file systems, networks, development teams, etc.)?

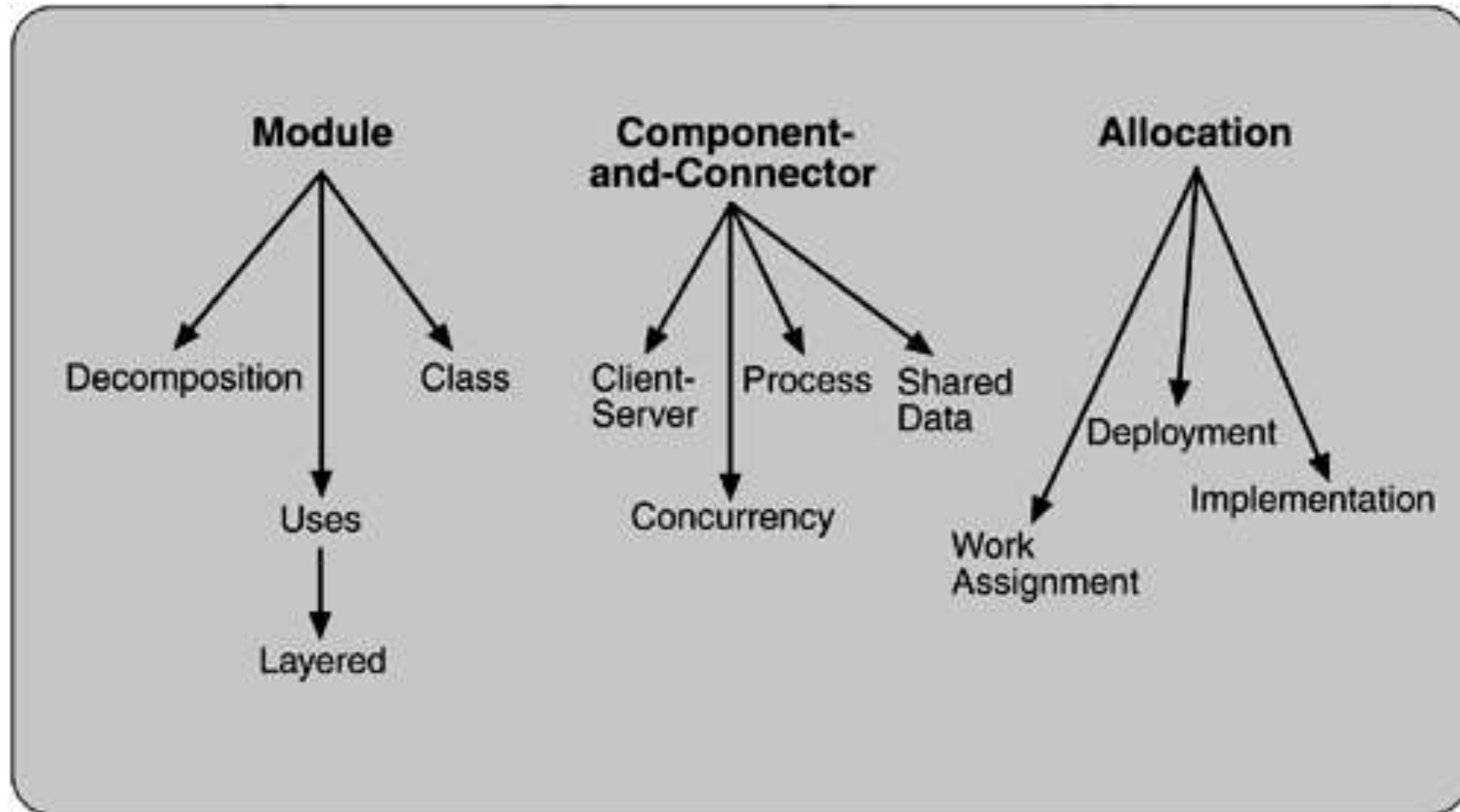


SOFTWARE STRUCTURES

3 Perspectives



SOFTWARE STRUCTURES





BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Module

Decomposition.



- The units are modules related to each other by the "is a submodule of " relation, showing how larger modules are decomposed into smaller ones recursively until they are small enough to be easily understood.
- Modules in this structure represent a common starting point for design, as the architect enumerates what the units of software will have to do and assigns each item to a module for subsequent (more detailed) design and eventual implementation.
- Modules often have associated products (i.e., interface specifications, code, test plans, etc.).
- The decomposition structure provides a large part of the system's modifiability, by ensuring that likely changes fall within the purview of at most a few small modules.
- It is often used as the basis for the development project's organization, including the structure of the documentation, and its integration and test plans. The units in this structure often have organization-specific names.
- Certain U.S. Department of Defense standards, for instance, define Computer Software Configuration Items (CSCIs) and Computer Software Components (CSCs), which are units of modular decomposition.

Uses.



- The units of this important but overlooked structure are also modules, or (in circumstances where a finer grain is warranted) procedures or resources on the interfaces of modules.
- The units are related by the uses relation.
- One unit uses another if the correctness of the first requires the presence of a correct version (as opposed to a stub) of the second.
- The uses structure is used to engineer systems that can be easily extended to add functionality or from which useful functional subsets can be easily extracted.
- The ability to easily subset a working system allows for incremental development.

Layered.



- When the uses relations in this structure are carefully controlled in a particular way, a system of layers emerges, in which a layer is a coherent set of related functionality.
- In a strictly layered structure, layer n may only use the services of layer $n - 1$.
- Many variations of this (and a lessening of this structural restriction) occur in practice, however.
- Layers are often designed as abstractions (virtual machines) that hide implementation specifics below from the layers above, engendering portability.

Class, or generalization.



- The module units in this structure are called classes.
- The relation is "inherits-from" or "is-an-instance-of."
- This view supports reasoning about collections of similar behavior or capability (i.e., the classes that other classes inherit from) and parameterized differences which are captured by subclassing.
- The class structure allows us to reason about re-use and the incremental addition of functionality.



Component and Connector

Process, or communicating processes.



- Like all component-and-connector structures, this one is orthogonal to the module-based structures and deals with the dynamic aspects of a running system.
- The units here are processes or threads that are connected with each other by communication, synchronization, and/or exclusion operations.
- The relation in this (and in all component-and-connector structures) is attachment, showing how the components and connectors are hooked together.
- The process structure is important in helping to engineer a system's execution performance and availability.

Concurrency.



- This component-and-connector structure allows the architect to determine opportunities for parallelism and the locations where resource contention may occur.
- The units are components and the connectors are "logical threads."
- A logical thread is a sequence of computation that can be allocated to a separate physical thread later in the design process.
- The concurrency structure is used early in design to identify the requirements for managing the issues associated with concurrent execution.

Shared data, or repository.



- This structure comprises components and connectors that create, store, and access persistent data.
- If the system is in fact structured around one or more shared data repositories, this structure is a good one to illuminate.
- It shows how data is produced and consumed by runtime software elements, and it can be used to ensure good performance and data integrity.

Client-server.



- If the system is built as a group of cooperating clients and servers, this is a good component-and-connector structure to illuminate.
- The components are the clients and servers, and the connectors are protocols and messages they share to carry out the system's work.
- This is useful for separation of concerns (supporting modifiability), for physical distribution, and for load balancing (supporting runtime performance).



BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Allocation

Deployment.



- The deployment structure shows how software is assigned to hardware-processing and communication elements.
- The elements are software (usually a process from a component-and-connector view), hardware entities (processors), and communication pathways.
- Relations are "allocated-to," showing on which physical units the software elements reside, and "migrates-to," if the allocation is dynamic.
- This view allows an engineer to reason about performance, data integrity, availability, and security.
- It is of particular interest in distributed or parallel systems.

Implementation.



- This structure shows how software elements (usually modules) are mapped to the file structure(s) in the system's development, integration, or configuration control environments.
- This is critical for the management of development activities and build processes.

Work assignment.



- This structure assigns responsibility for implementing and integrating the modules to the appropriate development teams.
- Having a work assignment structure as part of the architecture makes it clear that the decision about who does the work has architectural as well as management implications.
- The architect will know the expertise required on each team.
- Also, on large multi-sourced distributed development projects, the work assignment structure is the means for calling out units of functional commonality and assigning them to a single team, rather than having them implemented by everyone who needs them.

elements and relations in each structure



Software Structure	Relations	Useful for
Decomposition	Is a submodule of; shares secret with	Resource allocation and project structuring and planning; information hiding, encapsulation; configuration control
Uses	Requires the correct presence of	Engineering subsets; engineering extensions
Layered	Requires the correct presence of; uses the services of; provides abstraction to	Incremental development; implementing systems on top of "virtual machines" portability
Class	Is an instance of; shares access methods of	In object-oriented design systems, producing rapid almost-alike implementations from a common template

elements and relations in each structure



Software Structure	Relations	Useful for
Client-Server	Communicates with; depends on	Distributed operation; separation of concerns; performance analysis; load balancing
Process	Runs concurrently with; may run concurrently with; excludes; precedes; etc.	Scheduling analysis; performance analysis
Concurrency	Runs on the same logical thread	Identifying locations where resource contention exists, where threads may fork, join, be created or be killed
Shared Data	Produces data; consumes data	Performance; data integrity; modifiability

elements and relations in each structure



Software Structure	Relations	Useful for
Deployment	Allocated to; migrates to	Performance, availability, security analysis
Implementation	Stored in	Configuration control, integration, test activities
Work Assignment	Assigned to	Project management, best use of expertise, management of commonality

Case Study

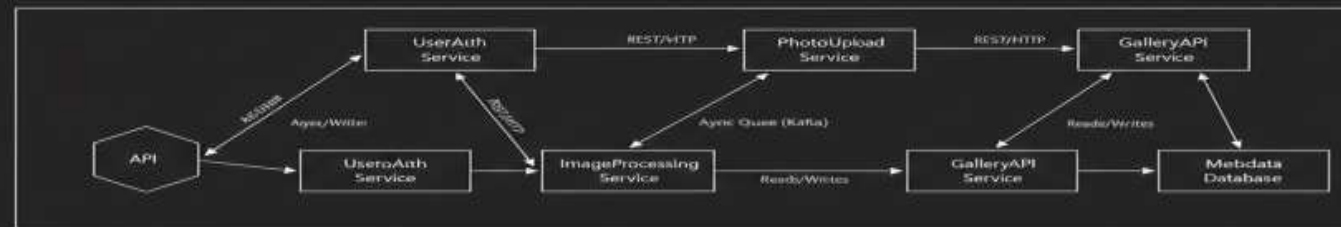


Case Study: Online Sharing Platform – 1. Module Structure View



Focus: Code organization, dependencies, and decomposition. For developers and maintainers.

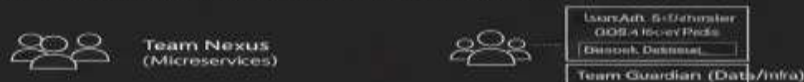
2. Component-and-Connector Structure View UML Package Diagram



Focus: Runtime components, data flow, concurrency, and performance.

For integrations and operations.

Focus: Runtime components, data flow, and performance, and ownership.



Focus: Deployment to hardware/cloud, infrastructure structure, data residency, and team object ownership.
For system engineers and project managers.





Kruchten's four views

WHICH STRUCTURES TO CHOOSE?



Logical.

The elements are "key abstractions," which are manifested in the object-oriented world as objects or object classes. This is a module view.

Process.

This view addresses concurrency and distribution of functionality. It is a component-and-connector view.

Development.

This view shows the organization of software modules, libraries, subsystems, and units of development. It is an allocation view, mapping software to the development environment.

Physical.

This view maps other elements onto processing and communication nodes and is also an allocation view (which others call the deployment view).



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Documenting Architecture Views

Harvinder S Jabbal
SSZG653 Software Architectures



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SEZG651/ SSZG653

Software Architectures

Module 4-CS 06/2

August 29, 2026

SSZG653 Software Architectures



Architecture Views



- Representation of a coherent set of architectural elements , as written by and read by system stakeholders.

Documenting



- “Documenting an architecture is a matter of documenting the relevant **views** and then adding a documentation that applies to more than one view.”

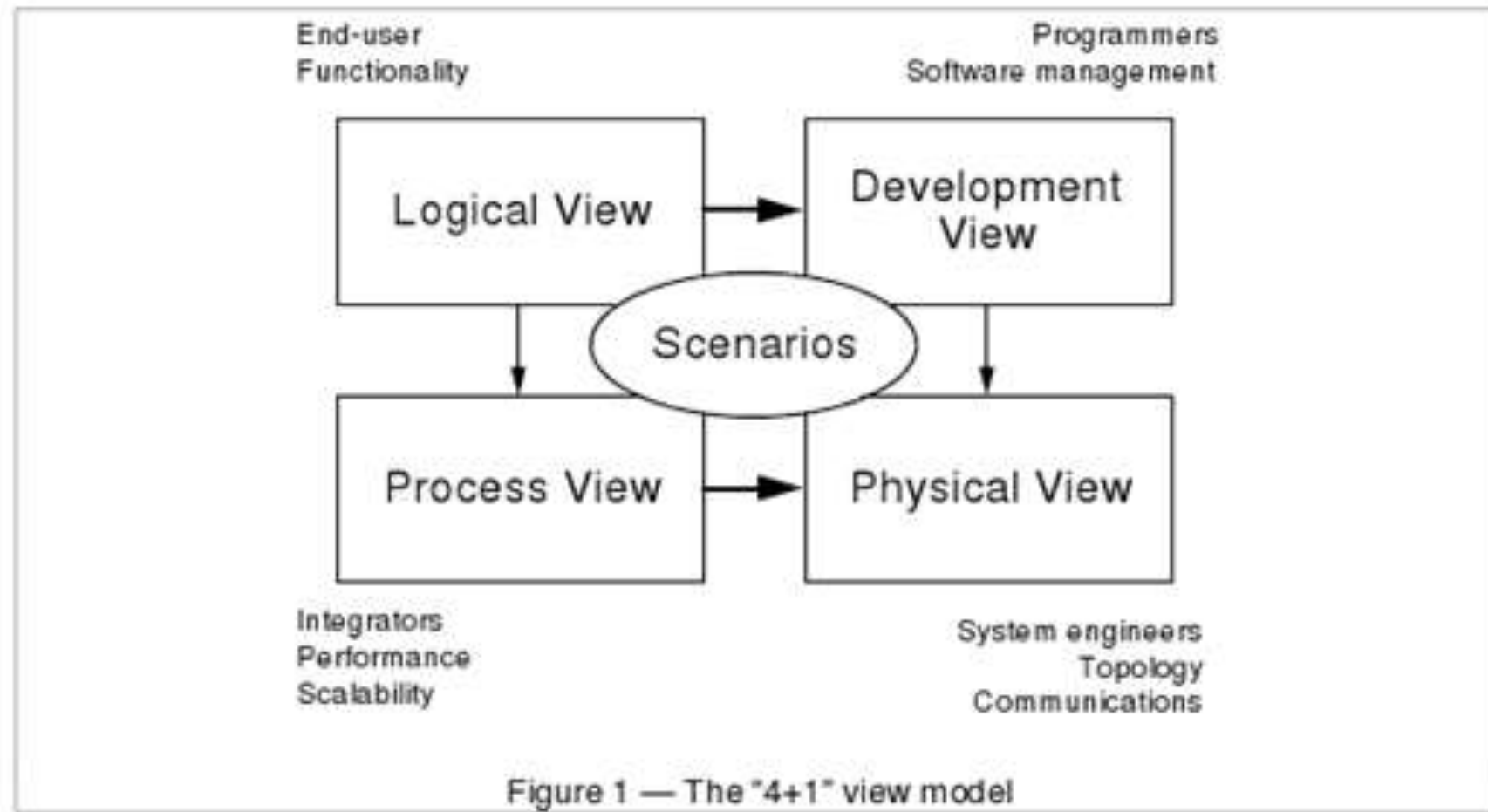
Solution to a Problem



- Problem
 - Architecture documents do not address the concerns of all stakeholders .
 - Deferent Stakeholders : end-user, system engineers, developers and project managers.
 - Architecture documents contained complex diagrams some times they are hard to be represented on the documentation.
- Solution
 - Using different notations for several **Views** each one addressing one specific set for concerns.

4+1 Model -

Philippe Kruchten, Rational Software Corp.



Logical View



- **The logical view**, which is the object model of the design (when an object-oriented design method is used)

Viewer: End-user

considers: Functional requirements- What are the services must be provided by the system to the users.

Notation: The Booch notation .

Tool: Rational Rose

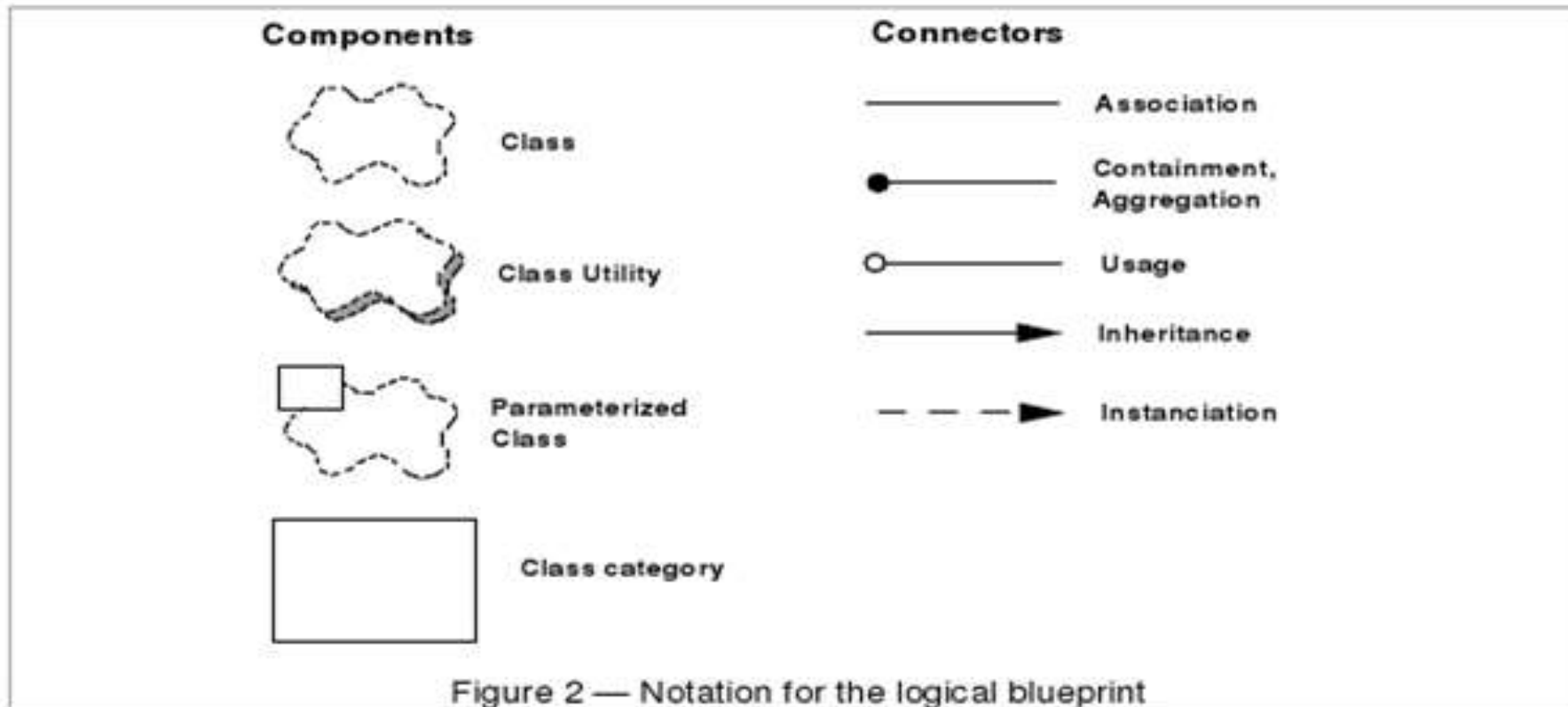
Notation for Logical View-

Philippe Kruchten, Rational Software Corp.



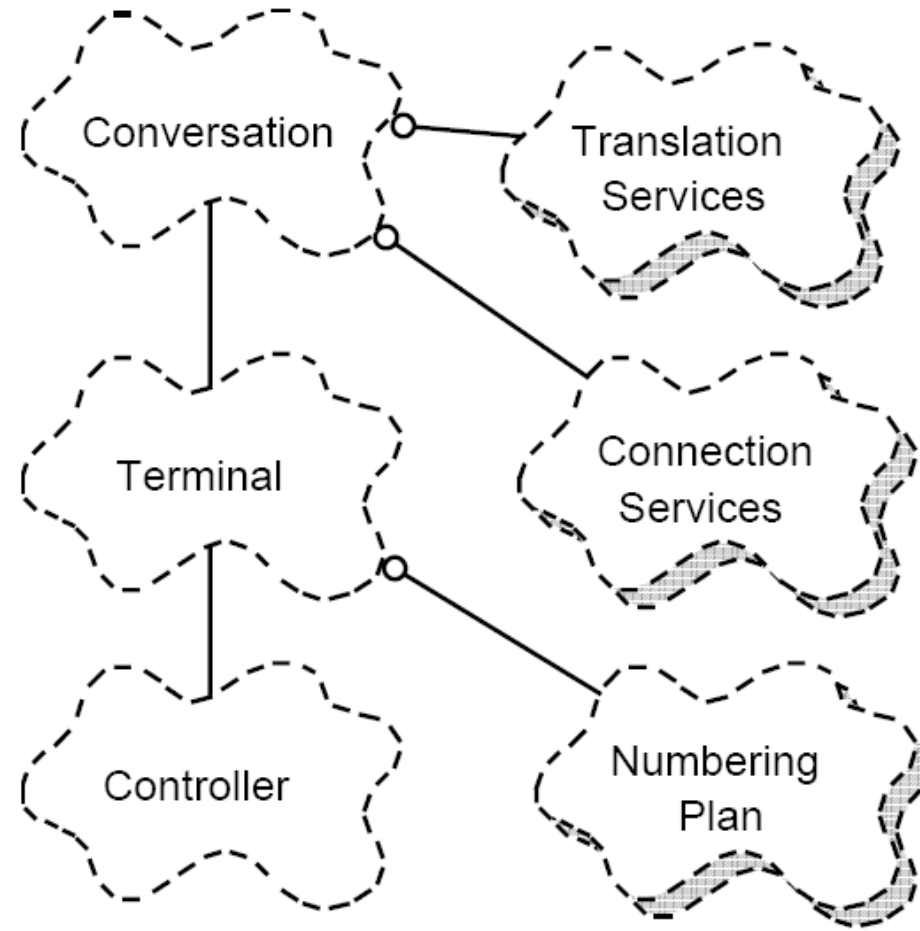
Notation for the logical view

The notation for the logical view is derived from the Booch notation⁴. It is considerably simplified to take into account only the items that are architecturally significant. In particular, the numerous adornments are not very useful at this level of design. We use Rational Rose[®] to support the logical architecture design.



Logical view Example PABX-

Philippe Kruchten, Rational Software Corp.



Process View



The process view, which captures the concurrency and synchronization aspects of the design(**The process decomposition**).

viewer: Integrators

considers: Non - functional requirements (scalability, concurrency, and performance)

style: Garlan and Shaw 's Architecture styles.

Process (cont.)



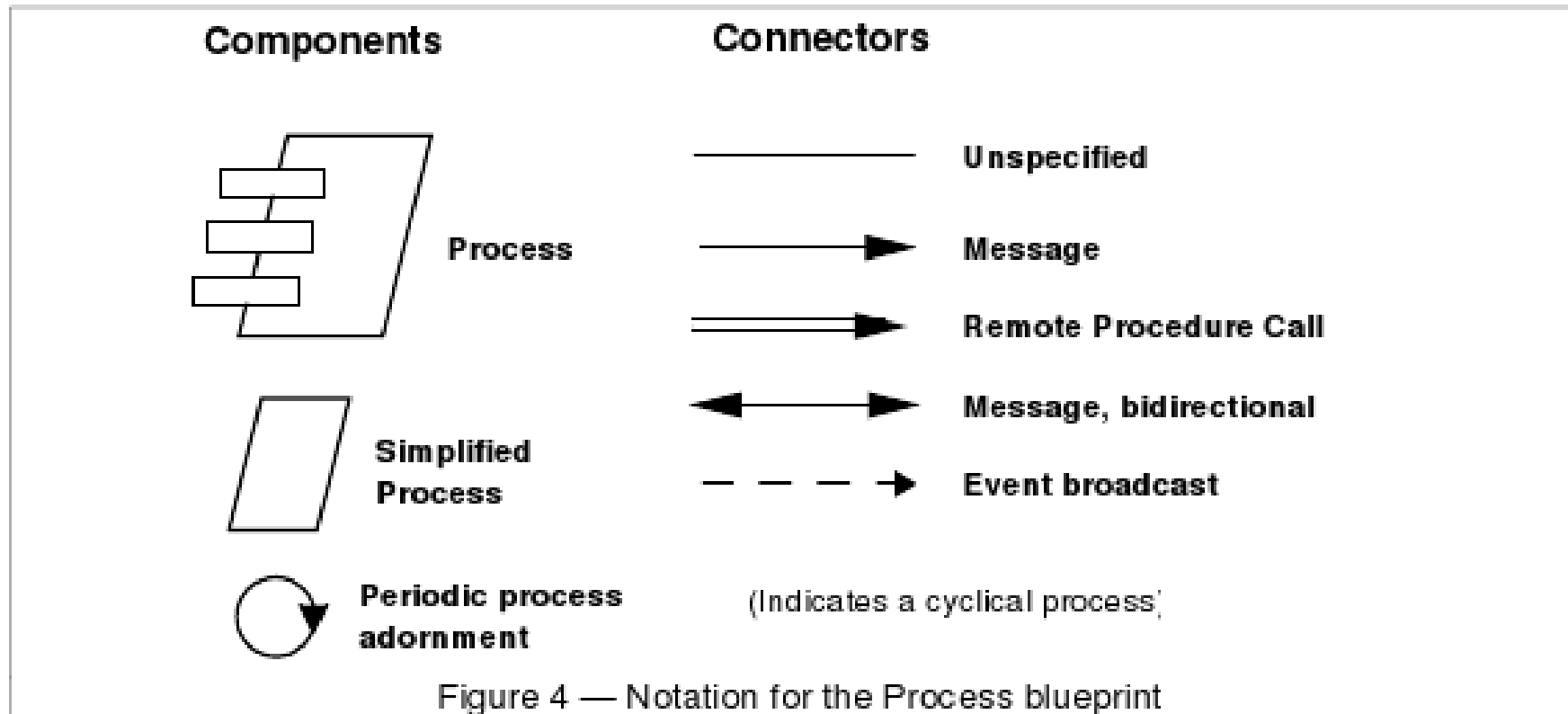
Uses multiple levels of abstractions.

A process is a grouping of tasks that form an executable unit:

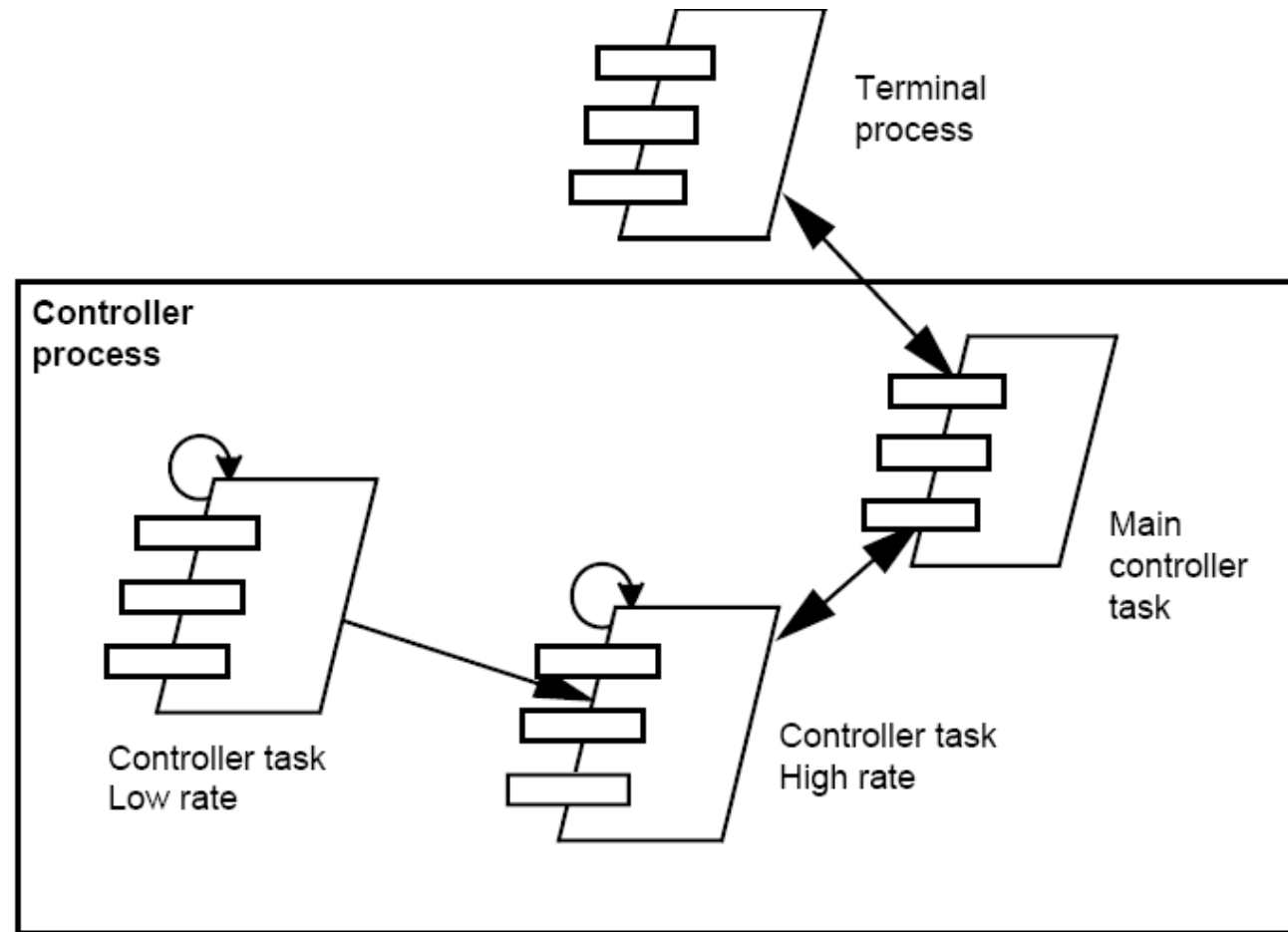
- Major Tasks: Architecture relevant tasks.
- Minor or helper Tasks: (Buffering)

Notation-

Philippe Kruchten, Rational Software Corp.



Process View example PABX (partial)- *Philippe Kruchten, Rational Software Corp.*



Development View



The *development view*, which describes the **static** organization of the software in its development environment.

Viewer: Programmers and Software Managers

considers: software module organization.

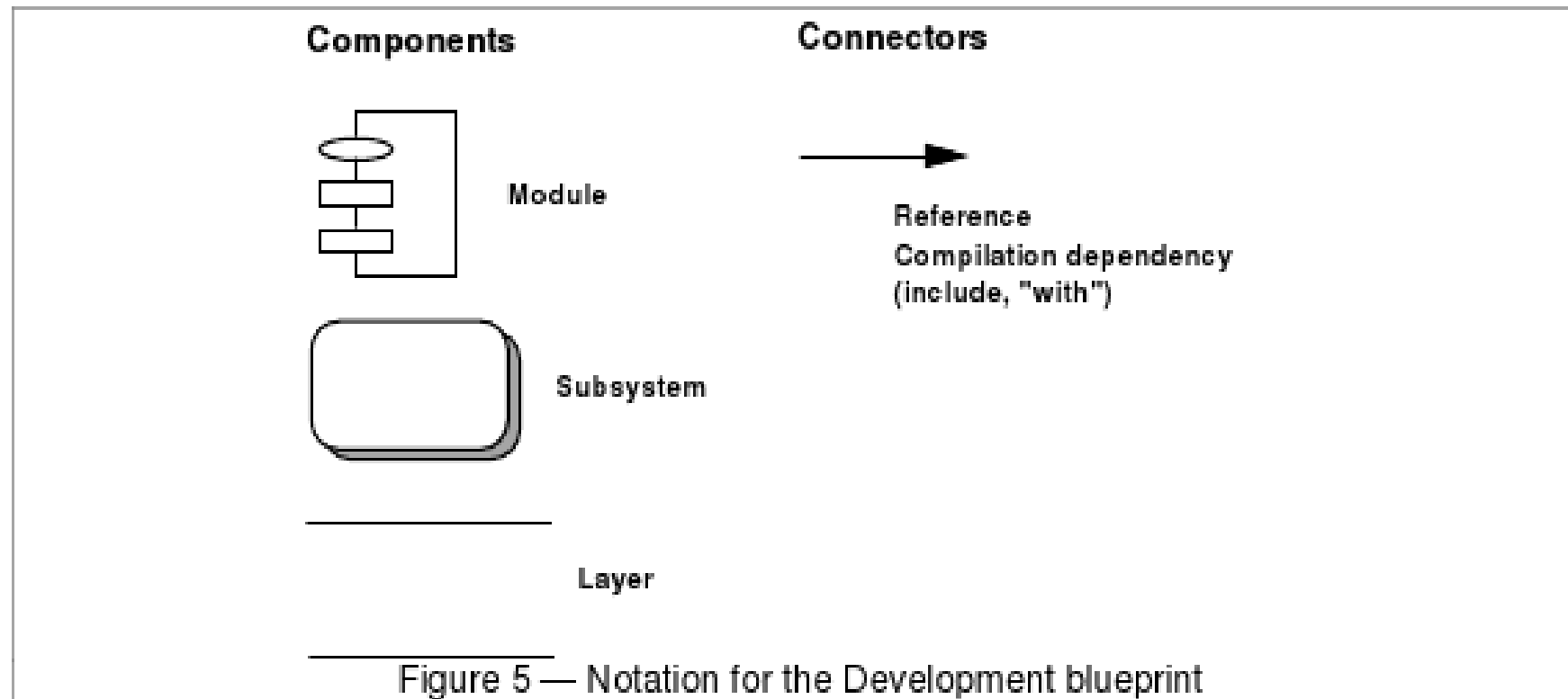
(Hierarchy of layers, software management, reuse, constraints of tools).

Notation: the Booch notation.

Style: layered style

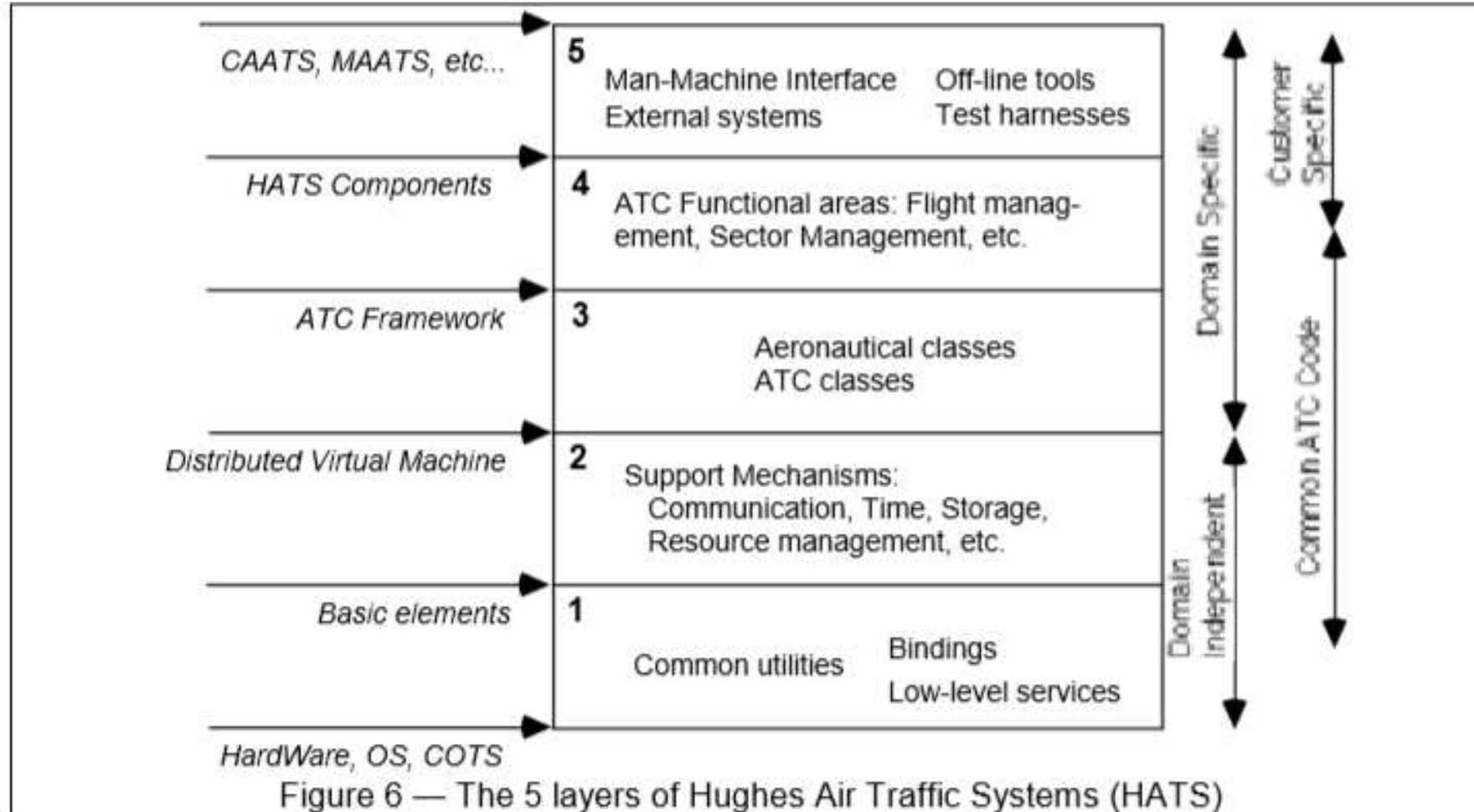
Notation for Development blueprint-

Philippe Kruchten, Rational Software Corp.



Development View – Layered Style

Philippe Kruchten, Rational Software Corp.



Physical View



the physical view, which describes the mapping(s) of the software onto the hardware and reflects its distributed aspect.

Viewer: System Engineers

Considers: Non-functional requirement (reliability, availability and performance). regarding to underlying hardware.

There may be two architecture:

- Test and development
- deployment

Notation for Physical view-

Philippe Kruchten, Rational Software Corp.

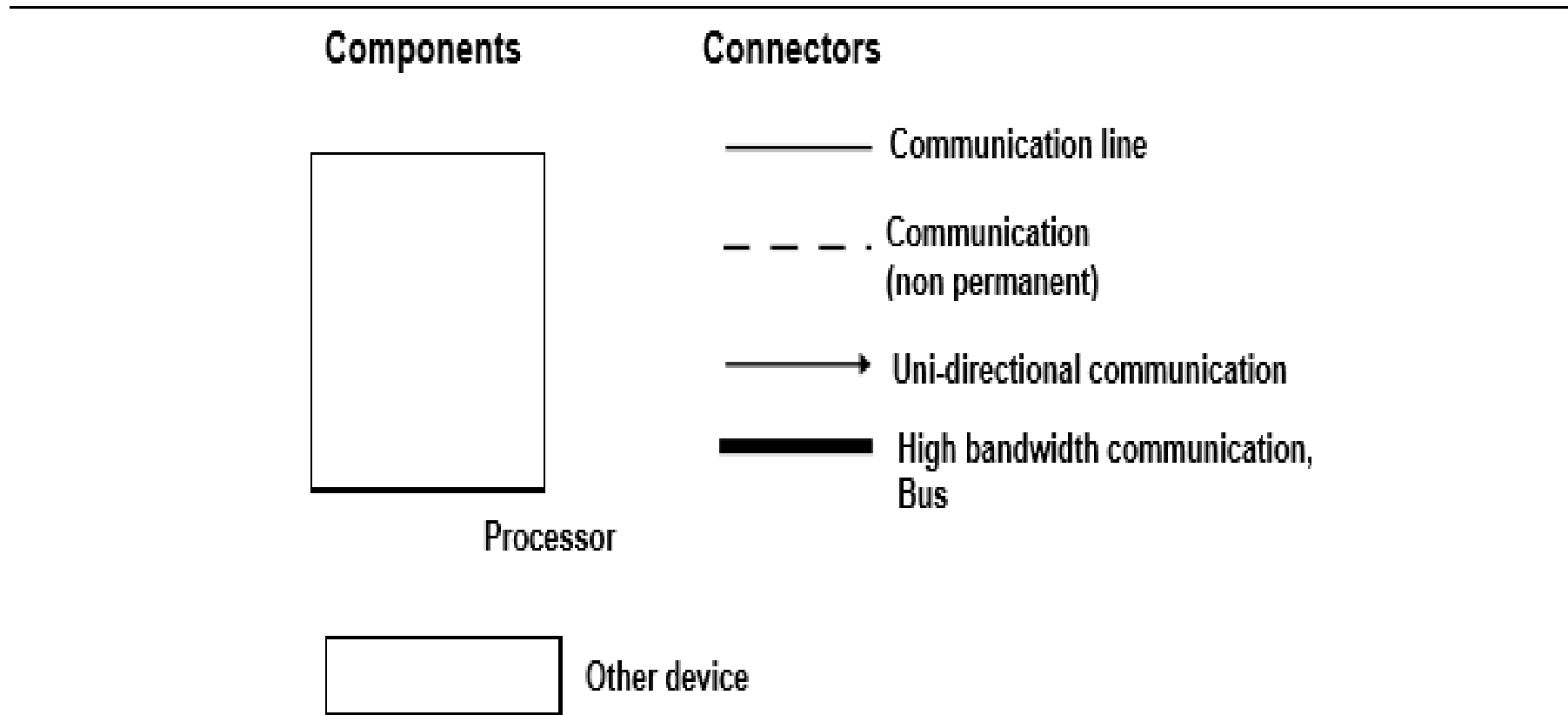


Figure 7 — Notation for the Physical blueprint

Physical blueprint PABX-

Philippe Kruchten, Rational Software Corp. (PHYSICAL AUTOMATIC BRANCH EXCHANGE)

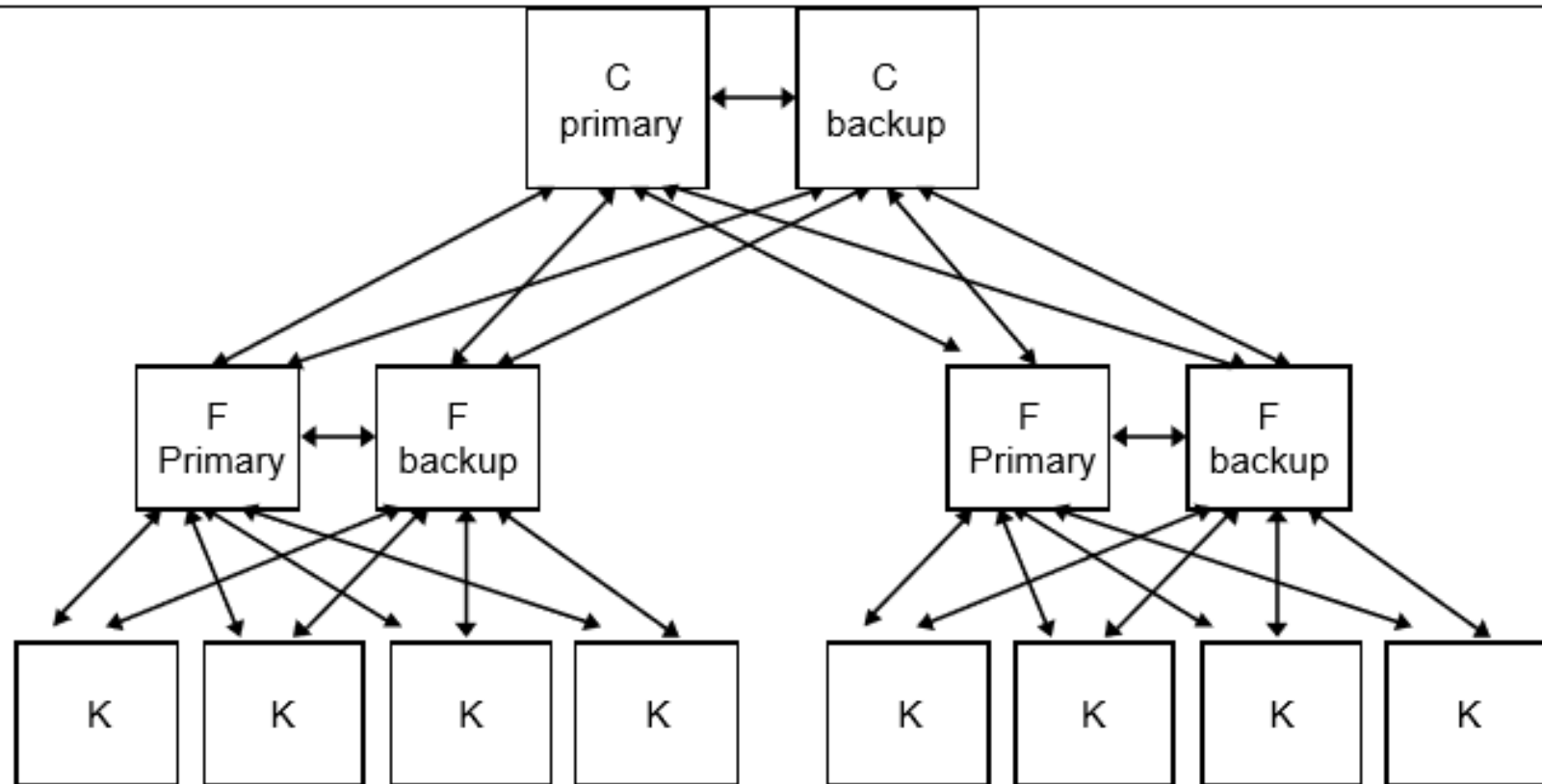
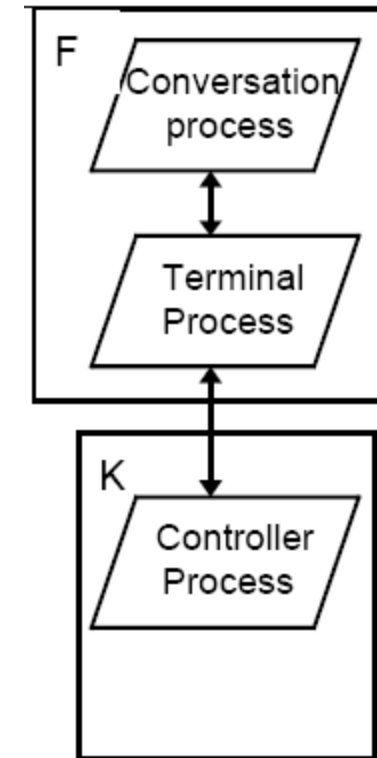


Figure 8 — Physical blueprint for the PABX

Physical view example-

Philippe Kruchten, Rational Software Corp.



A small PABX physical architecture with process allocation.

Physical view example-

Philippe Kruchten, Rational Software Corp.

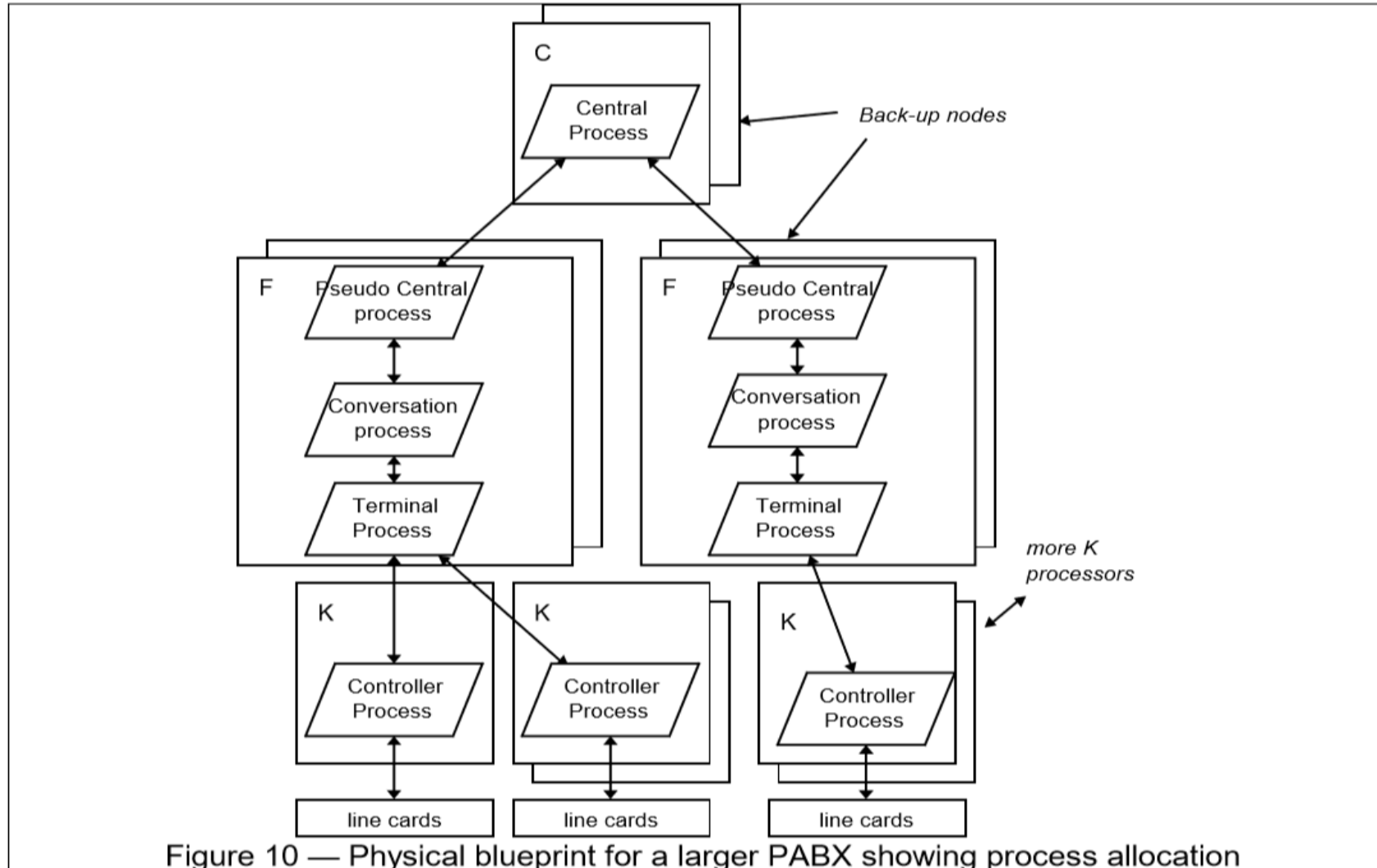


Figure 10 — Physical blueprint for a larger PABX showing process allocation

Scenarios



(Putting all “4 views” together)

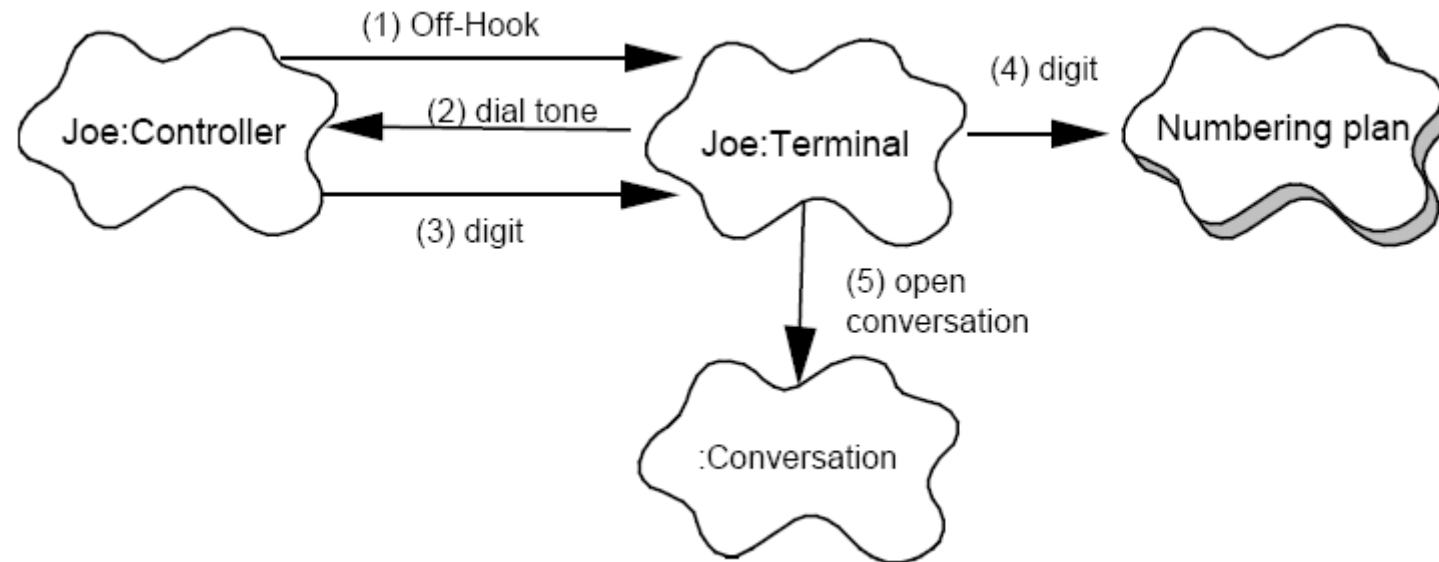
Viewer: All users and Evaluators.

Considers: System consistency and validity

Notation: Similar to logical view

Scenario example-

Philippe Kruchten, Rational Software Corp.



Scenario for a Local call – selection phase

Correspondence between the views



The **views** are interconnected.

Start with Logical view and Move to Development / Process view and then finally go to Physical view.

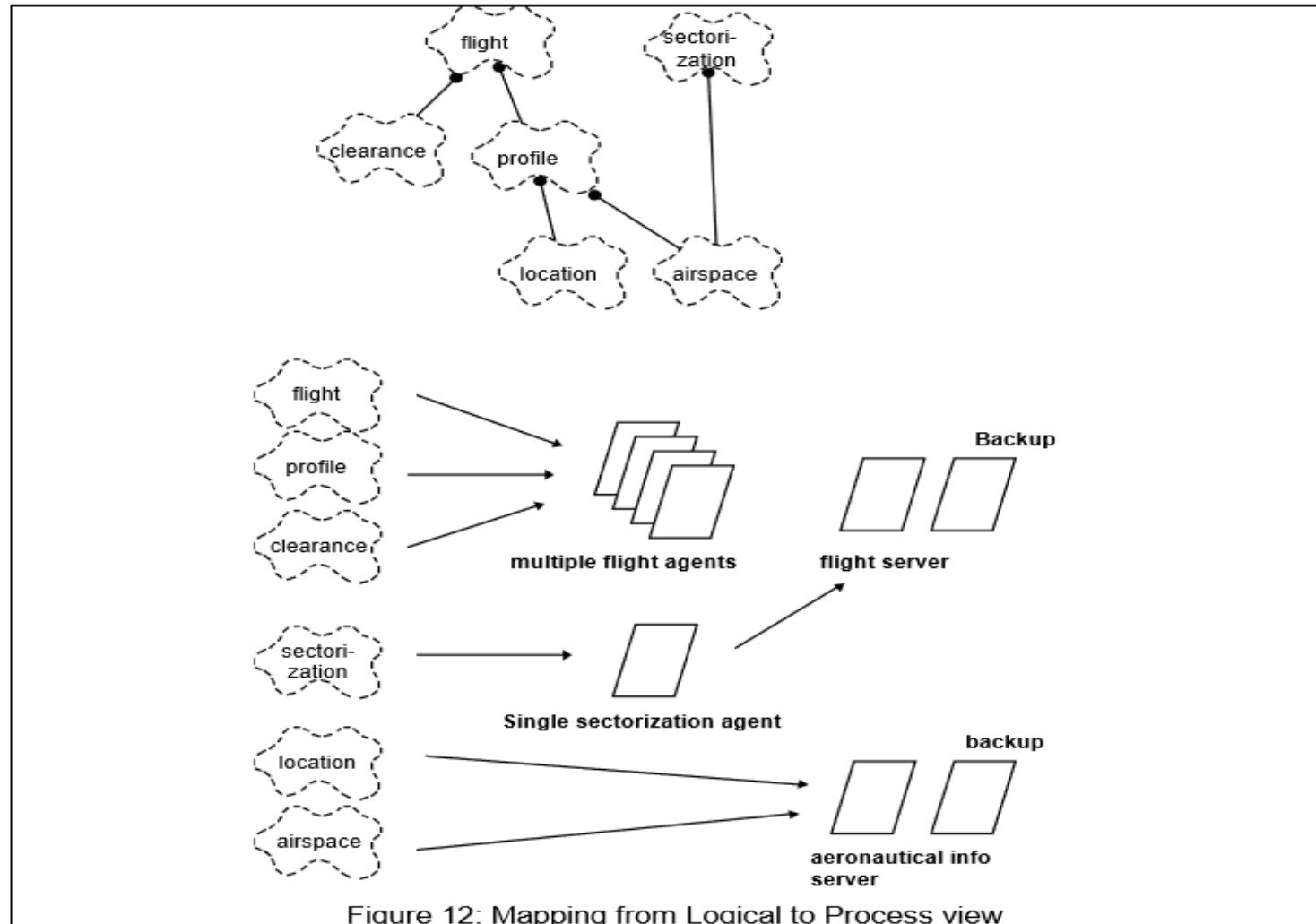
From logical to Process view



Two strategies :

- Inside-out: starting from Logical structure
- Outside-in: starting from physical structure

From logical to Process view



From Logical to development



They are very close, but the larger the project, the greater the distance between these views.

Grouping to subsystems depending on:

- The team organization.
- The class categories which includes the packages.
- The Line of codes.

Iterative process



Not all architectures need all views.

A scenario-driven approach to develop the system is used to handle the iterative.

Documenting the architecture:

- **Software architecture document:** follows closely “4+1” views.
- **Software design guidelines:** it captured the most important design decisions that must be respected to **maintain** the architectural **integrity**.

Software Design Document



Title Page
Change History
Table of Contents
List of Figures
1. Scope
2. References
3. Software Architecture
4. Architectural Goals & Constraints
5. Logical Architecture
6. Process Architecture
7. Development Architecture
8. Physical Architecture
9. Scenarios
10. Size and Performance
11. Quality
Appendices
A. Acronyms and Abbreviations
B. Definitions
C. Design Principles

Figure 13 — Outline of a Software Architecture Document

Annotation:



- “4+1 views” methodology successfully used in the industry
 - Air Traffic Control
 - Telecom
- This paper missing the tools to integrate these views which lead to an inconsistency problem.
- The inconsistency problem is more tangible in the maintenance of the architecture.

Summary



<i>View</i>	<i>Logical</i>	<i>Process</i>	<i>Development</i>	<i>Physical</i>	<i>Scenarios</i>
<i>Components</i>	Class	Task	Module, Subsystem	Node	Step, Scripts
<i>Connectors</i>	association, inheritance, containment	Rendez-vous, Message, broadcast, RPC, etc.	compilation dependency, "with" clause, "include"	Communication medium, LAN, WAN, bus, etc.	
<i>Containers</i>	Class category	Process	Subsystem (library)	Physical subsystem	Web
<i>Stakeholders</i>	End-user	System designer, integrator	Developer, manager	System designer	End-user, developer
<i>Concerns</i>	Functionality	Performance, availability, S/W fault-tolerance, integrity	Organization, reuse, portability, line-of-product	Scalability, performance, availability	Understandability
<i>Tool support</i>	Rose	UNAS/SALE DADS	Apex, SoDA	UNAS, Openview DADS	Rose

Table 1 — Summary of the "4+1" view model

Synthesis: Mapping SEI Structures to 4+1 Views



Content: While the terminology differs, the underlying architectural intent is the same. Understanding this mapping prevents "documentation silo"

SEI Structure Category	Kruchten 4+1 View	Primary Focus
Module	Logical View	Functional decomposition, classes, and layers.
Component & Connector	Process View	Runtime behavior, throughput, and scalability.
Allocation	Development View	Mapping software to the file system/CM environment.
Allocation	Physical View	Mapping software to hardware/cloud nodes.

Key takeaway: Use SEI definitions for what to document, and 4+1 for who (stakeholders) you are documenting for.

Modernizing the Logical View



Logical View: Beyond Booch to UML 2.5 **Content:**

The Shift: While the original 4+1 papers used Booch notation, modern architecture uses **UML (Unified Modeling Language)**.

Key Diagrams to Include:

- **Class Diagrams:** To show static relationships and "is-a" hierarchies.
- **Package Diagrams:** To show high-level module grouping and encapsulation.

Modern Context: In a Microservices world, the Logical View defines the **Service API contracts** and domain boundaries (Bounded Contexts).

The Evolution of the Process View



Process View: Concurrency in the Cloud Era **Content:**

From PABX to Microservices: Modern components are often independent containers (Docker) or Serverless functions (Lambda).

Dynamic Behavior: * Sequence Diagrams: Essential for showing how components exchange messages over time.

- **Activity Diagrams:** To model complex business workflows and parallel processing.

Concerns: Focus on latency, deadlocks, and "Eventual Consistency" in distributed systems.

Development View & Modern Tooling



Development View: The "Build" Environment **Content:**

Scope: This view must now account for **DevOps** and **CI/CD** pipelines.

Key Elements:

- **Repository Structure:** Monorepo vs. Multi-repo strategies.
- **Third-party Dependencies:** Managing NuGet/NPM/Maven packages.

Stakeholders: Primarily for Developers and Build Engineers to ensure the system is "buildable" and "testable."

Physical View: Hardware to Infrastructure as Code (IaC)



Physical View: Deployment in the 2020s **Content:**

The Virtualized Reality: Physical nodes are rarely "bare metal" today; they are Virtual Machines (VMs) or Clusters (Kubernetes).

Key Concepts:

- **Elasticity:** How the architecture scales up/down automatically.
- **Network Topologies:** Firewalls, API Gateways, and Load Balancers.

Documentation Tool: Use **Deployment Diagrams** to map software artifacts to execution environments.

Maintaining Architectural Integrity



The Challenge of Architectural Drift **Content:**

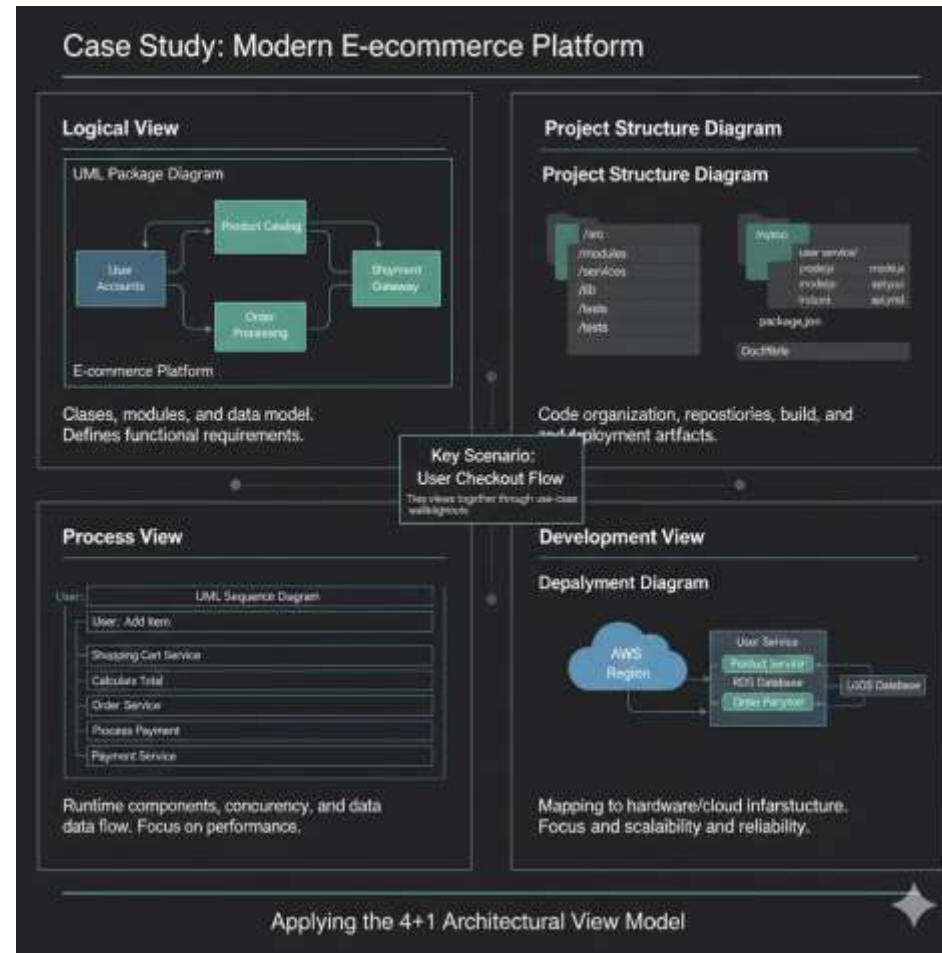
The Problem: Over time, the code (Implementation) evolves faster than the documentation (Views). This is known as **Architectural Drift**.

The Solution:

- **Architecture as Code:** Using tools like *Mermaid.js* or *PlantUML* to keep diagrams in the same Git repo as the code.
- **Automated Checks:** Using "fitness functions" to ensure code dependencies don't violate the Layered structure defined in the Module View.

Goal: The architecture should be a "living document," not a static PDF.

CASE STUDY



Summary & Stakeholder Mapping



Summary: Choosing the Right View **Content:** Not every project needs every view. Use this guide to prioritize:

Building a complex UI? Prioritize the **Logical View**.

High-performance/High-traffic? Prioritize the **Process View**.

Large, distributed team? Prioritize the **Development View**.

Cloud-native/Complex security? Prioritize the **Physical View**.

Conclusion: Document only what is necessary to communicate the design and mitigate risk.